

Documenting Human vs. AI Contributions

1. AI (Gemini) Contribution and Role

1. **Problem Formulation and Core Strategy:** The initial analytical goal of building a **mean-reversion pairs trading strategy** using **Z-Score analysis** was a human decision. This required understanding the non-stationary nature of price series and selecting OLS regression to establish the hedge ratio (β) and the Augmented Dickey-Fuller (ADF) test to validate spread stationarity—the fundamental prerequisites for a reliable statistical arbitrage strategy.
2. **Architecture Design:** The decision to use a **three-tier asynchronous pipeline** utilizing **PostgreSQL** as the central data bus was a core architectural choice to guarantee data integrity and separate the slow I/O of web sockets from the fast rendering of the Streamlit UI.
3. **Data Integrity and Robustness:** The developer manually identified and prioritized the **zero-price corruption issue** caused by flawed tick data. The fix—implementing a **MIN_PRICE_THRESHOLD filter** in `websocket_test.py`—was a vital analytical decision to maintain data quality for the analytics engine, preventing the corruption of all subsequent calculations (Hedge Ratio, Spread).
4. **Solving Data Gaps (Gap-Filling Logic):** After observing gapped candles during system downtime, the developer reasoned that the fixed 60-minute lookback in the generator was insufficient. The solution—dynamically querying the database for the **timestamp of the last saved candle** and processing all ticks since that time—demonstrates strong debugging and domain-specific knowledge of incremental data processing.
5. **UX Optimization (Flicker Fix):** The recognition that Streamlit's mandatory full re-execution was destroying the UX was a key insight. The developer chose to implement **Streamlit Fragments (@st.experimental_fragment)** to isolate dynamic plotting and analysis, solving the flickering problem while keeping the performance benefits of caching. This required deep knowledge of Streamlit's architecture.
6. **Feature Integration and Scope Management:** The strategic decision to expand the system from 2 to **12 symbols** was handled efficiently by updating configuration lists rather than restructuring the core analytical code, proving scalability. The integration of **Telegram alerts** was a human choice to bridge the dashboard data to an actionable external notification system.

2. AI (Gemini) Contribution and Role

1. **Code Generation and Formatting:** The AI was primarily used to generate complete, clean Python classes (e.g., the initial drafts of `PairsAnalytics` and the `AlertManager`) based on functional requirements provided by the developer. This saved significant time in manual coding and syntax verification.
2. **Debugging Assistance:** The AI was instrumental in diagnosing the exact error in specific code blocks.
3. **Structural Guidance:** The AI provided the technical blueprint for achieving complex goals, such as the initial structure of the **Telegram API integration** and the scaffolding for the **Streamlit Fragment implementation**, ensuring correct decorator usage and proper integration with existing class methods.
4. **Verification and Review:** The AI acted as a continuous reviewer, running cross-checks and verifying that new code additions (like the 12-symbol update) did not regress or break previously working features, significantly accelerating the testing and quality assurance phase.